

TTDS 2024 Past Paper Deep Solutions

Original-question guide + worked solutions

Generated for TTDS revision

2026-05-20

Contents

使用方式	1
Question 1: Preprocessing, Crawling, Permuterm, Compression	1
Question 2: IR Evaluation	2
Question 3: tf-idf, Jaccard, LM, Rocchio	4
Question 4: Expected Mutual Information	6
Question 5: Manual Content Analysis and Automation	7
Question 6: Topic Modelling and Classification Short Answers	8
Question 7: Query Logs and Click-through Data	8
Coverage Checklist	9

使用方式

这份解析对应 2024462_INFR11229.pdf。重点是 TTDS 经典 calculation questions: IR evaluation、tf-idf、language modelling、Rocchio、mutual information。所有关键术语保留英文。

Question 1: Preprocessing, Crawling, Permuterm, Compression

原题要点

考 stemming、removing non-alphanumeric characters、web crawling、permuterm index、politeness vs freshness、v-byte decoding + delta decoding。

(a) Preprocessing

Stemming

Definition: reduce inflected/derived words to a common stem, e.g. connect, connected, connection -> connect.

Advantage: improves recall by matching morphological variants.

Drawback: can reduce precision through overstemming, e.g. universe and university may be conflated by aggressive stemmers.

Removing non-alphanumeric characters

Definition: remove punctuation, symbols, emojis, URLs fragments, etc.

Advantage: reduces noise and vocabulary size, useful for clean news-style corpora.

Drawback: can destroy useful meaning, e.g. C++, COVID-19, hashtags, email addresses, emoticons, or sentiment punctuation.

(b) How web search engines crawl

Web crawlers start from seed URLs, fetch pages, parse links, normalise URLs, add unseen URLs to a URL frontier, obey robots.txt and politeness constraints, revisit pages for freshness, and store downloaded pages for indexing.

(c) Permuterm Index

Permuterm index supports wildcard queries by storing all rotations of each term plus an end marker \$.

Example for hello\$: hello\$, ello\$h, llo\$he, lo\$hel, o\$hell, \$hello.

Main drawback: dictionary size blows up because each term of length m creates $m + 1$ rotations.

Wildcard lookup:

- *tion -> rotate to tion\$, look up prefix tion\$
- blo*k -> add \$: blo*k\$; rotate so wildcard at end -> k\$blo*, look up prefix k\$blo

(d) Politeness vs Freshness

Politeness means not overloading servers; freshness means revisiting pages often enough to keep the index up to date. Balance them by per-host crawl delays/back queues, priority scores, and adaptive revisit frequency: fast-changing important pages get revisited more often, but same-host requests are spaced out.

(e) v-byte and delta decoding

Sequence:

10010010 00100011 00100100 10000001 00001001 10000101

Using lecture convention: high bit 1 terminates a number, high bit 0 continues, remaining 7 bits are data.

Groups:

- 10010010 -> data 0010010 = 18
- 00100011 00100100 10000001 -> data 0100011 0100100 0000001 = 578049
- 00001001 10000101 -> data 0001001 0000101 = 1157

Delta/gap list:

[18, 578049, 1157]

Original postings list, assuming first posting unchanged:

[18, 18 + 578049, 578067 + 1157] = [18, 578067, 579224]

常见坑

v-byte 先解出 gaps, 不是直接得到 postings list。恢复 postings list 要做 cumulative sum。

Question 2: IR Evaluation

原题要点

Systems S1-S5 return top 20. Relevant docs are A-H. Need:

- Precision at R=50%
- R@6
- AP
- F1@10
- largest possible MAP/AP for S2
- nDCG@10 for S2 with graded relevance
- highest nDCG@1 and nDCG@2

Relevant ranks

System	Relevant ranks
S1	1, 2, 5, 7, 8, 11, 14, 15
S2	2, 4, 5, 6, 11, 13
S3	1, 3, 5, 6, 11, 13, 16, 20
S4	1, 2, 4, 6, 8, 13, 18, 19
S5	1, 3, 4, 5, 13, 16, 19, 20

(a) Metrics and rankings

Precision at $R=50\%$ means precision when recall first reaches 50%. There are 8 relevant docs, so recall 50% means retrieving 4 relevant docs.

System	P at R=50%	R@6	AP
S1	0.571	0.375	0.672
S2	0.667	0.500	0.398
S3	0.667	0.500	0.586
S4	0.667	0.500	0.664
S5	0.800	0.500	0.593

Rankings:

- P at R=50%: $S5 > S2 = S3 = S4 > S1$
- R@6: $S2 = S3 = S4 = S5 > S1$
- AP: $S1 > S4 > S5 > S3 > S2$

AP formula:

$$AP = \frac{1}{R} \sum_{k=1}^n P@k \cdot rel(k)$$

(b) F1@10

Top 10 relevant counts:

- S1: 5 relevant $\rightarrow P = 5/10 = 0.5, R = 5/8 = 0.625, F1 = 0.556$
- S2: 4 relevant $\rightarrow P = 0.4, R = 0.5, F1 = 0.444$
- S3: 4 relevant $\rightarrow F1 = 0.444$
- S4: 5 relevant $\rightarrow F1 = 0.556$
- S5: 4 relevant $\rightarrow F1 = 0.444$

System	F1@10
S1	0.556
S2	0.444
S3	0.444
S4	0.556
S5	0.444

(c) Largest possible AP/MAP for S2

The table says top 20 only, but each system returned the whole collection. For S2, relevant docs shown are E, H, A, C, D, F: 6 out of A-H. Missing relevant docs B and G could appear immediately at ranks 21 and 22 in the best case.

Then S2 relevant ranks are:

2, 4, 5, 6, 11, 13, 21, 22

$$AP_{max} = \frac{1}{8} \left(\frac{1}{2} + \frac{2}{4} + \frac{3}{5} + \frac{4}{6} + \frac{5}{11} + \frac{6}{13} + \frac{7}{21} + \frac{8}{22} \right) = 0.485$$

For a single query, MAP equals AP. So largest possible MAP for S2 on this query is 0.485.

(d) nDCG@10 for S2

Graded relevance: A=B=3, C=D=E=2, F=G=H=1.

S2 top 10:

N, E, N, H, A, C, N, N, N, N

Gains:

[0, 2, 0, 1, 3, 2, 0, 0, 0, 0]

Using:

$$DCG@10 = \sum_{i=1}^{10} \frac{gain_i}{\log_2(i+1)}$$

$$DCG@10 = 3.566$$

Ideal gains:

[3, 3, 2, 2, 2, 1, 1, 1, 0, 0]

$$IDCG@10 = 8.533$$

$$nDCG@10 = 3.566/8.533 = 0.418$$

(e) Highest nDCG@1 and nDCG@2

For nDCG@1, only first document matters. S3 has A at rank 1, gain 3, so S3 has nDCG@1 = 1. S1/S4/S5 have gain 2 at rank 1, so 0.667. S2 has N, so 0.

Highest nDCG@1: **S3**.

For nDCG@2, S1 ranks D,A gains [2,3], S4 ranks E,C gains [2,2], S5 ranks C,N gains [2,0], S3 ranks A,N gains [3,0], S2 ranks N,E gains [0,2]. Highest is **S1**.

Question 3: tf-idf, Jaccard, LM, Rocchio

原题要点

Query: brown fox. 8 documents. Use base 10 logs, raw tf, no document length normalisation.

Documents:

Doc	Text
d1	brown fox brown fox
d2	run watch fox jump
d3	fat brown fat fox
d4	go fox brown go
d5	jump brown brown fox
d6	brown brown brown jump
d7	watch jump run fox

Doc	Text
d8	nice quite lovely fox

(a) Highest idf

$$idf(t) = \log_{10} \frac{N}{df(t)}, \quad N = 8$$

Terms with $df = 1$ have highest idf:

fat, go, nice, quite, lovely

$$idf = \log_{10}(8/1) = 0.903$$

(b) Jaccard(d3, d5)

Use sets of terms:

- d3 = {fat, brown, fox}
- d5 = {jump, brown, fox}

Intersection = {brown, fox}, size 2. Union = {fat, jump, brown, fox}, size 4.

$$Jaccard = \frac{2}{4} = 0.500$$

(c) Language modelling: $P(fat|d3)$ with Jelinek-Mercer

$$P(t|d) = \lambda P_{MLE}(t|d) + (1 - \lambda)P(t|C)$$

$\lambda = 0.5$.

In d3, fat appears 2 times out of 4:

$$P_{MLE}(fat|d3) = 2/4 = 0.5$$

Collection has 32 tokens total; fat appears 2 times:

$$P(fat|C) = 2/32 = 0.0625$$

$$P(fat|d3) = 0.5(0.5) + 0.5(0.0625) = 0.281$$

(d) tf-idf ranking for query brown fox

Relevant idf values:

$$idf(brown) = \log_{10}(8/5) = 0.204$$

$$idf(fox) = \log_{10}(8/7) = 0.058$$

Score:

$$score(d, q) = tf(brown, d)idf(brown) + tf(fox, d)idf(fox)$$

Doc	Score
d6	0.612
d1	0.524
d5	0.466
d3	0.262
d4	0.262
d2	0.058
d7	0.058
d8	0.058

Ranking:

$$d6 > d1 > d5 > d3 = d4 > d2 = d7 = d8$$

(e) Rocchio feedback

User marks d7 relevant and d1 non-relevant. Set $\alpha = \beta = \gamma = 1$.

$$\vec{q}' = \vec{q}_0 + \vec{d}_{rel} - \vec{d}_{nonrel}$$

Initial query vector:

$$\vec{q}_0 = \{brown : 0.204, fox : 0.058\}$$

d7 = watch jump run fox:

$$\vec{d}_7 = \{watch : 0.602, jump : 0.301, run : 0.602, fox : 0.058\}$$

d1 = brown fox brown fox:

$$\vec{d}_1 = \{brown : 0.408, fox : 0.116\}$$

Thus:

Term	New weight
brown	-0.204
fox	0.000
jump	0.301
run	0.602
watch	0.602
all other terms	0.000

Interpretation: feedback moves the query away from d1' s repeated brown fox and toward d7' s terms watch, run, jump. Negative weights are sometimes clipped in practical systems.

Question 4: Expected Mutual Information

原题要点

Compute MI for apple with Tech, and llama with Tech, then compare frequency vs MI.

Totals:

$$N = 2303 + 1361 + 803 + 1403 + 650 + 801 = 7321$$

(a) apple and Tech

For apple:

- $N_{11} = 310$
- $N_{1.} = 56 + 156 + 310 + 23 + 2 + 41 = 588$
- $N_{.1} = 803$
- $N_{10} = 278$
- $N_{01} = 493$
- $N_{00} = 6240$

Contributions:

Cell	Contribution
N_{11}	0.095910
N_{01}	-0.039261
N_{10}	-0.034674
N_{00}	0.049357

$$I(\text{apple}; \text{Tech}) = 0.071$$

(b) llama and Tech

For llama:

- $N_{11} = 320$
- $N_{1.} = 452 + 24 + 320 + 41 + 3 + 3 = 843$
- $N_{.1} = 803$
- $N_{10} = 523$
- $N_{01} = 483$
- $N_{00} = 5995$

Contributions:

Cell	Contribution
N_{11}	0.078289
N_{01}	-0.036740
N_{10}	-0.037227
N_{00}	0.045712

$$I(\text{llama}; \text{Tech}) = 0.050$$

(c) Comparison

llama occurs slightly more often in Tech documents: 320 vs apple 310. But apple has higher MI: 0.071 vs 0.050.

Reason: MI measures association, not just raw within-class frequency. llama appears often in Animals too, so it is less specifically associated with Tech. apple is more distinctive for Tech relative to its overall distribution.

Question 5: Manual Content Analysis and Automation

(a) Manual content analysis

Manual content analysis means researchers define a coding scheme/categories, sample documents, train human coders, annotate documents, check inter-annotator agreement, refine coding guidelines, then analyse counts/patterns.

(b) When automation makes sense

Example: classifying millions of tweets as customer complaints vs praise. It makes sense because scale is high, labels are relatively consistent, and small errors may be acceptable for aggregate analysis.

(c) When automation does not make sense

Example: high-stakes legal judgement or subtle qualitative interpretation of political rhetoric. It may not make sense because context is complex, labels require expert judgement, and errors have serious consequences.

Question 6: Topic Modelling and Classification Short Answers

(a) Unigram model vs mixture of unigrams

Unigram model assumes all words in a document are generated from one word distribution. Mixture of unigrams assumes each document first chooses one latent topic/component, then all words are generated from that topic's word distribution. It captures document-level clustering but not multiple topics per document.

(b) Why LDA generative process is unrealistic

LDA assumes each word's topic is sampled independently from the document-topic distribution, ignoring syntax and word order. Example: in "the bank raised interest rates", words are not independent; grammar and collocations matter. LDA is useful as a bag-of-words topic model, not a realistic model of human text production.

(c) OOV and transfer learning

i. OOV words

Out-of-vocabulary words are words appearing at test time that were not in the training vocabulary.

ii. Traditional solution

Use feature pruning with an <UNK> token, character n-grams, subword features, or retrain vocabulary with more data.

iii. Why less issue with transfer learning

Modern pretrained models often use subword tokenisation such as BPE/WordPiece, so unseen words can be decomposed into known subword units. They also bring pretrained semantic representations.

(d) Zero-shot classification

Zero-shot classification means classifying examples into labels without task-specific labelled training examples for those labels. It is useful when labelled data is scarce, classes change often, or we want rapid deployment using natural-language label descriptions.

Question 7: Query Logs and Click-through Data

原题要点

Colleague has hand-crafted ranking features but wants improvement. Name three ways query logs/click-through can help.

Answer

1. Query expansion / suggestion

Logs reveal common reformulations and related queries. If users search "car insurance" then "cheap auto insurance", the system can learn query similarity.

2. Learning-to-rank features

Clicks, dwell time, skips, and reformulations can become implicit relevance signals. These train or tune ranking models beyond hand-crafted features.

3. Detect popular intents and navigational targets

For frequent queries, click-through identifies preferred URLs or dominant user intent. This improves ranking for head queries and navigational queries.

Caveats

Click data is biased by rank position and presentation. A top result gets more clicks even if not best, so logs need debiasing or careful interpretation.

Coverage Checklist

Question	Covered TTDS topics
Q1	Preprocessing, stemming, URL/symbol handling, crawling, politeness/freshness, permuterm index, v-byte/delta encoding
Q2	Precision at recall, recall@k, AP/MAP, F1, nDCG, graded relevance
Q3	tf-idf, idf, Jaccard, language modelling, Jelinek-Mercer smoothing, Rocchio relevance feedback
Q4	Expected mutual information, feature selection, contingency tables
Q5	Manual content analysis, automated classification, when ML is appropriate
Q6	Unigram model, mixture of unigrams, LDA assumptions, OOV, transfer learning, zero-shot classification
Q7	Query logs, click-through data, implicit feedback, learning to rank